

Protocolos de comunicacion

Jaime Nazar Anchorena

Contents

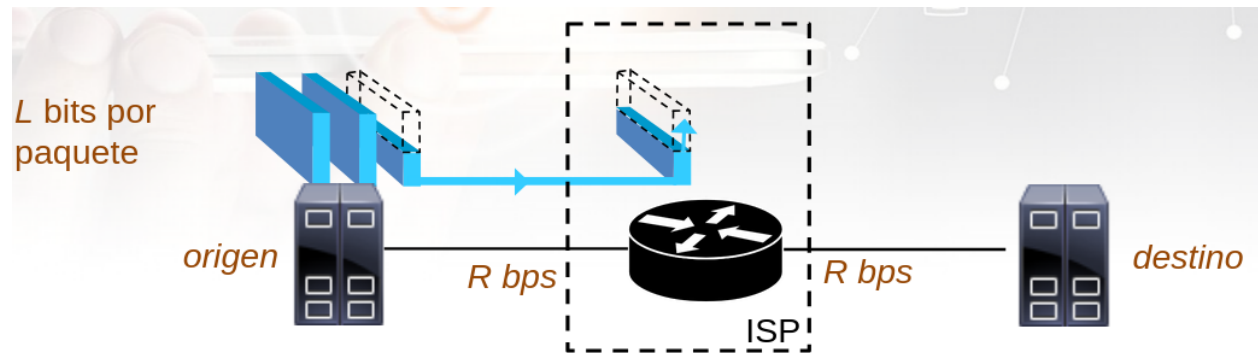
Unidad 1: Introduccion	3
Modelo OSI - TCP/IP	4
Capa de enlaces	4
Capa de ruteo	5
Capa de transporte: Puestos	5
Capa de aplicacion: soporte a aplicaciones	5
Serializacion	5
Unidad 2: HTTP	5
HTTP 0.9(1991)	5
HTTP 1.0(1996)	6
HTTP 1.1(1999)	6
HTTP/2	6
HTTP/3	6
Recursos	6
URI(RFC 3986 3.3)	6
URL	6
URN	7
Request menos utilizados	7
Respuestas	7
Tipos de informacion	7
GET vs POST	8
Headers HTTP	8
Tipos	8
Headers generales	8
Headers de solicitud	8
Headers de respuesta	9
Headers de contenido	9
Proxy servers	9
Proxy reverso	9
Cookies	11
Curl	11
Resolucion de nombres - DNS	11
Como se consiguen los nombres	12

Cache	13
Configuracion de servidor DNS	13
Registros DNS	13
Formato RR	13
Split-horizon	14
DNS dinamico	14
DNS spoofing	14
Metodo: Enviar respuesta sin recibir pregunta	14
Metodo: DNS sniffer	14
Metodo: DNS cache poisoning	14

Unidad 1: Introduccion

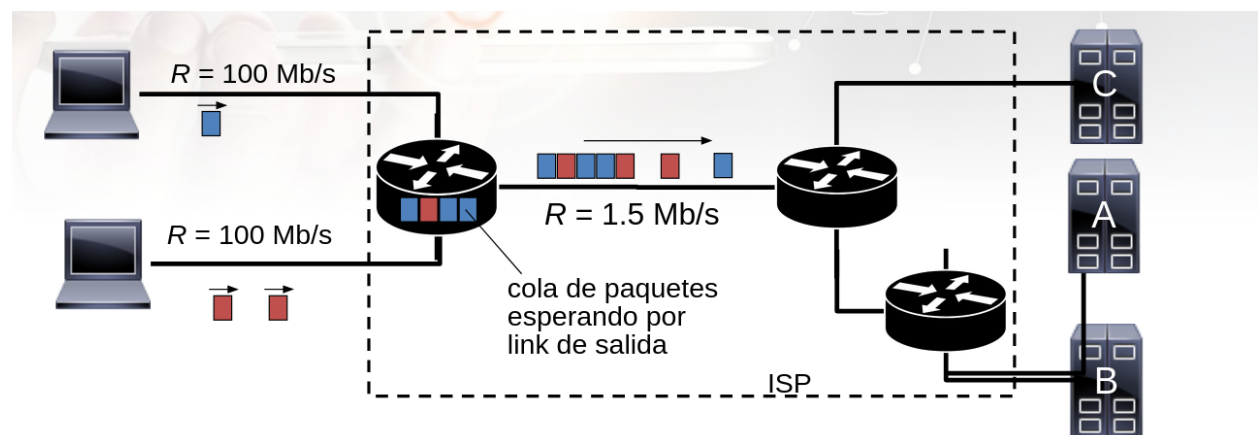
Las redes que vamos a ver es un conjunto interconectado de hosts, cada host es un equipo autonomo y no son necesariamente computadoras(pensar en IoT). El host envia datos en paquetes, de L bits de longitud.

Hay un problema, por ejemplo cuando contratamos un ISP, ya que nos venden la velocidad entre nuestra casa al ISP, pero no garantiza la misma velocidad a otros servicios. Ademas, si todos se conectan al mismo ISP, aparece un cuello de botella, haciendo que todo sea mas lento.



$$\text{Demora de transmisión de un paquete} = \text{tiempo necesario para transmitir paquete de } L \text{ bits} = \frac{L \text{ (bits)}}{R \text{ (bits/sec)}}$$

store and forward: el paquete debe arribar al router en forma completa antes de ser retransmitido



encolamiento y pérdida:

- ❖ Si la tasa del link de entrada es superior a la de transmisión en un determinado período:
 - se encolan paquetes, esperando por ser transmitidos
 - si el buffer está lleno, paquetes entrantes se "droppean"

Antes, tampoco habia estandares, entonces sistemas de diferentes empresas podian no ser compatible.

Modelo OSI - TCP/IP

Es un modelo de 7 capas, usando el protocolo TCP/IP, el cual se utilizo para estandarizar todo esto.

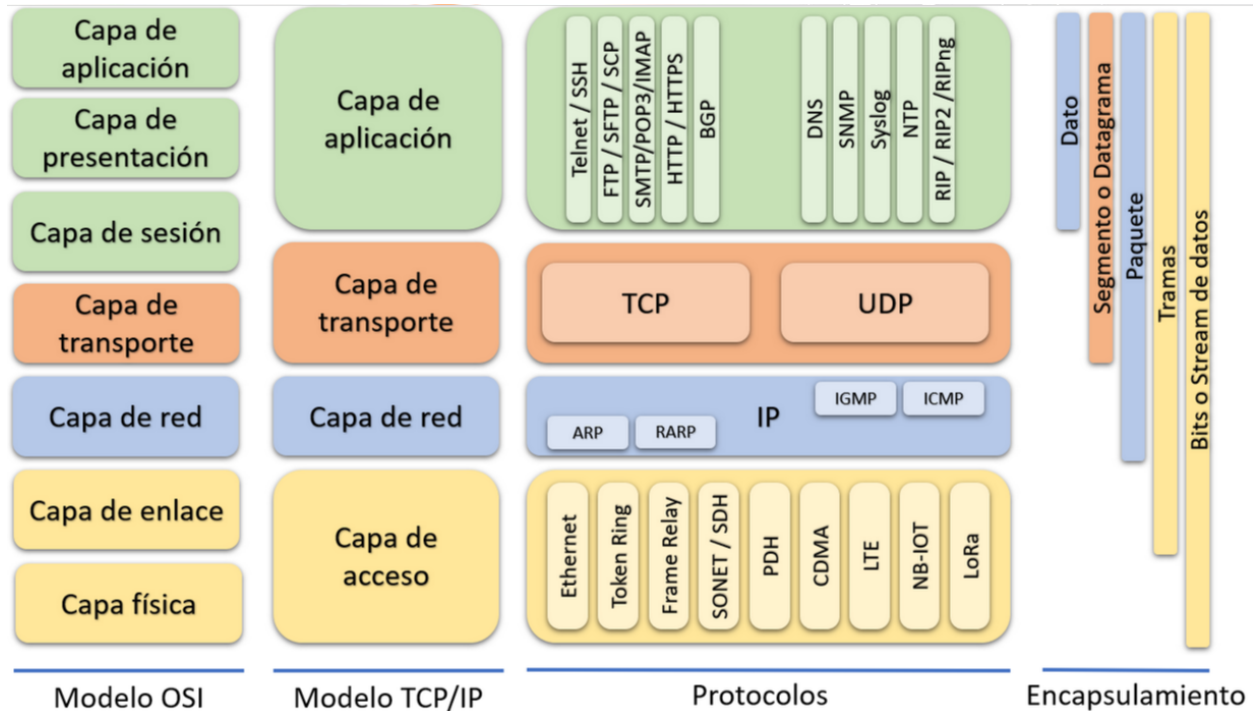


Figure 1: Modelo OSI - TCP/IP

A la capa de red no le importa el motivo, solo le importa hacer llegar un paquete de una computadora a la otra.

Un protocolo es un conjunto de reglas que ambas puntas tienen que seguir

Estos protocolos pueden ser:

- **Confiables:** Confirma si la información fue recibida, utiliza acuse de recibo, reenvía información de ser necesario e informa al nivel superior si no se pudo enviar
- **No confiable:** No puede asegurar si el destinatario recibió o no la información enviable

Capa de enlaces

Tiene varios objetivos:

- Vincular hosts contiguos
- Controlar el acceso al medio físico
- Detectar y/o corregir errores básicos
- Encapsular los paquetes en tramas

Por ejemplo, una topología tipo **bus**. Para evitar colisiones, surgieron dos aparatos:

- **Hub:** Menos inteligente, trabaja en la capa física

- **Switch:** Mas inteligente, trabaja en la capa de enlace. Asocia las MAC de las vocas(va aprendiendo).

Capa de ruteo

Hay que distribuir las IP alrededor del mundo, entonces, se dividio en zonas para que cada una realiza la ditribucion. No se pueden conectar dos host que usan enlaces diferentes, para ello se necesita un router. El router de nuestras casas tiene 2 **interfaces**, pues hay una interfaz por red(WAN y LAN). El router domiciliario es mucho mas que un simple router, pues internamente tiene un switch para el LAN que luego va a una de las interfaces. Sin embargo, a nivel capa de IP, todo esto es una misma red con la excepcion de WAN(red externa). Se podria decir que el switch tambien tiene 2 interfaces mas pues tiene 2.4GHz y 5.8GHz, entonces... tiene 6 interfaces el router? **No**, pues en un switch se llaman vocas(capa enlace), las interfaces son de la capa IP.

En conclusion, vamos a considerar que un router tiene dos interfecez, todo esto a nivel logico y de seguridad.

Capa de transporte: Puestos

El numero de puerto identifica a los procesos dentro de hosts.

Capa de aplicacion: soporte a aplicaciones

Aca aparecen los servicios mas conocidos: HTTP, POP3, SMTP, DNS, SSH, etc.

Comando que vamos a usar en todas las practicas: `tail -f /var/log/syslog` (-f es de follow)

Serializacion

- Se puede transmitir numeros entre computadoras? Si y no, dependera como codifica el numero(little endian o big endian) una computadora y como lo decodifica la otra. Para solucionar esto, en los protocolos de comunicacion se transmite todo en big endian. La mayoría de las computadoras hoy en dia usan little endian.
- Cuando tenemos que calcular la longitud de un string codificado en unicode, y tiene caracteres especiales, surgen problemas pues los caracteres podrian ocupas mas de un byte.

Unidad 2: HTTP

Antes, para obtener archivos se utilizaba un protocolo llamado **FTP**. En base a las referencias se debia acceder a otro sitio FTP y descargar el/los textos deseados. Surge la necesidad de aplicaciones basadas en **hipertextos**, para no tener que hacer todo manualmente, de ahi surge en HTTP.

HTTP 0.9(1991)

La primer version estaba basado en texto de recuperacion de informacion con un mecanismo request-response. Era contenido estatico y no mantiene el estado. Orientado a la **linea de comando**. Mas adelante se dieron cuenta que tenia potencial comercial, para que sirva para algo mas que papers cientificos.

TCP es de texto o binario? Es binario, mientras que HTTP es de texto.

HTTP 1.0(1996)

Ahora necesita ser **browser friendly**, entonces se agregan comandos como GET, POST, HEAD. Además, puede transmitir otras cosas que no sean hipertexto.

HTTP 1.1(1999)

Aparecen cabeceras en las peticiones, se agregaron los métodos PUT, DELETE, TRACE, OPTIONS. Negociación de contenido, soporte de cache, conexiones persistentes. Ahora cada recurso se identifica con una **URI**(Uniform Resource Identifier)

HTTP/2

Pasa a ser un protocolo binario, tiene compresión de headers, multiplexación de recursos, server push, etc. Es mucho más rápido que HTTP 1.1.

Comparación: <http://www.http2demo.io/>

Porque alguien usaría HTTP 1.1? Algun cliente que no soporte HTTP/2 o porque no hay soporte para la tecnología que tenemos que usar.

HTTP/3

Usa el protocolo **QUIC**(por ahí no lleguemos a verlo) como **transporte**, tiene menor latencia, multiplexación mejorada, etc. QUIC es un protocolo de transporte pensado para usar on HTTP pues pide muchos recursos. Desde el punto de vista de la funcionalidad, es de transporte, pero se podría considerar de aplicación.

HTTP/3 no usa TCP, usa QUIC

Recursos

Bloque de información identificado por su URI, puede ser un archivo(físico) o generado por un programa(abstracción). La URI puede ser un URL o un URN. Con URN el archivo puede cambiar de lugar y el servidor se va a encargar de buscarlo, mientras que con URL se pierde.

Que diferencia hay entre una URL y una URI? URI es la clase abstracta para identificar recursos de forma unívoca, mientras que URL y URN son clases concretas.

Los protocolos son públicos y se identifican con RFC(1630, 2396, 2718, 3305, 3986).

URI(RFC 3986 3.3)

`<scheme>://<authority><path>?<query>`

El path termina con el primer ? o # o si no hay más caracteres. Puede ser relativo o absolutos. Si representa una aplicación puede recibir parámetros. El schema identifica el protocolo y la mayoría de los browsers lo agregan por defecto. Pueden ser usadas para acceder recursos, ya sea por medio de URL o URN.

URL

`<scheme>://<user>:<password>@<host>:<port>/<path>?<query>`

Identifica un recurso por su ubicación.

URN

Identifica un recurso por su nombre, no implica que el recurso exista o como acceder a el.

Por ejemplo:

`urn:isbn:0132856204`

Escribimos `www.clarin.com` y el browser se encarga de detectar que es HTTP. Luego el browser averigua la IP(dsp lo vemos) y se conecta mediante TCP al servidor.

En caso(por ejemplo) de AWS, no es factible asignarle una IP publica a cada cliente, lo que se hace es tener todas en el mismo servidor(misma IP publica) y se diferencias las paginas por la URI, entonces, a veces puede no funcionar solicitar una pagina web escribiendo directamente la IP.

Por otro lado, los headers en un request estan delimitados por un espacio(`\n\n`), esto aprovecha que es texto, sino no se podria hacer. Mientras que para saber cuando termina el cuerpo se indica en el header con `content-length`.

Esto funciona perfecto si es estatico, pero surgen problemas si es dinamico.

Request menos utilizados

- PUT
- TRACE
- OPTION
- DELETE

Respuestas

- 1XX: Informacion
- 2XX: Exito
- 3XX: Redireccion
- 4XX: Error en cliente
- 5XX: Error en servidor

La diferencia entre **moved permanently**(301) y **found**(302), a pesar de que ambos solo redireccionan, es que el primero es permanente, no va a cambiar. Por lo tanto, uno es una regla que no va a cambiar, permitiendo que el navegador se lo guarde y lo use para no gastarse en mandar de vuelta el request.

Tipos de informacion

HTTP solo transmite texto, utiliza **MIME** para describir contenido multimedia, cada objeto es etiquetado por el web server. Se define en el header `accept`.

- `text/html`
- `text/plain`
- `video/mp4`
- `image/jpeg`
- `application/pdf`

- etc.

GET vs POST

La idea es que el **GET** sea idempotente. Puede ser cacheado, el browser los mantiene en el historial, pueden ser **bookmarked**, tienen longitud acotada y solo son para recibir datos.

Idempotente es que puedo hacer **GET** todas las veces que pueda y el estado del servidor seguirá siendo el mismo (PUT no es idempotente, puede cambiar el estado en el primer request y en el segundo el estado será otro).

DELETE es idempotente también.

Mientras que el **POST** nunca es cacheado, no se mantienen en el historial del browser, no pueden ser **bookmarked**, no tienen longitud de datos máxima.

Headers HTTP

Como se mencionó anteriormente, finaliza con una línea en blanco.

`<nombre>: <valor asociado>`

Tipos

- Generales
- De solicitudes
- De respuestas
- De contenido

Generalmente se envían en ese orden.

Headers generales

- **Cache-control:** Directivas para cache
- **Connection:** Se definen opciones de conexión
- **Date:** Fecha de creación del mensaje
- **Transfer-Encoding:** Indica el encoding de transferencia
- **Via:** Muestra la lista de intermediarios por los que pasó el mensaje.

Headers de solicitud

- **Accept:** Tipo de contenido aceptado por el cliente
- **Accept-Charset:** Charset (ISO-xxxx, UTF-8) aceptado por el cliente
- **Accept-Encoding:** Encoding (gzip, compress) aceptado por el cliente
- **Expect:** Comportamiento esperado del server frente al request.
- **From:** Email del usuario de la aplicación que generó el request.
- **Host:** Servidor y puerto destino del request.
- **If-Modified-Since:** Condiciona al request a la fecha indicada.
- **Referer:** URL del documento que generó el request

- **User-Agent:** Aplicación que generó el request
- **Upgrade:** solicita que use otro protocolo
- **Range:** solicita un rango (en bytes) del recurso

Referer esta mal escrito(va con *rr*), fue un error y quedo asi :/.

accept es lo que le mandamos al servidor, pero despues el servidor especifica cual te manda en **content-type**.

Headers de respuesta

- **Age:** Estimación en segundos del tiempo que fue generada la respuesta en el server.
- **Connection:** close, keep-alive
- **Location:** URI a redireccionar
- **Retry-After:** Tiempo de delay para reintento
- **Server:** Descripción del software del server.
- **Authorization:** indica que el recurso necesita autorización

Headers de contenido

- **Allow:** Métodos aplicables al recurso.
- **Content-Encoding**
- **Content-Length**
- **Content-Location**
- **Content-MD5**
- **Content-Type**
- **Expires:** Fecha de expiración del recurso.
- **Last-Modified:** Fecha de modificación del recurso

Proxy servers

Foward proxy, es algo que esta en el medio, pueded ser para molestar tambien. Actua como intermediario entre una plicacion cliente y un web server. Puede ser:

- **Explicito:** Lo configuramos nosotros, sabemos de su existencia.
- **Transparente:** A traves de un firewall, no nos damos cuenta que esta.

Tiene varias finalidades:

- Controlar el acceso a determinados sitios o paginas
- Almacenar en cache recientes consultas, haciendo que sean mas rapida pues usamos la red local

Sin embargo, el cache puede causar contenido desactualizado, por lo que el que genera el recurso decide cuanto tiempo dura el cache. Setea el parametro **max-age**.

Proxy reverso

El proxy directo esta del lado del cliente, mientras que el reverso, **reverse proxy** esta del lado del servidor. Redirecciona el trafico internamente a distintos servidores, por ejemplo, **NGINX**.

En el cache del browser/proxy esta **last-modified**, **max-age**, etc.

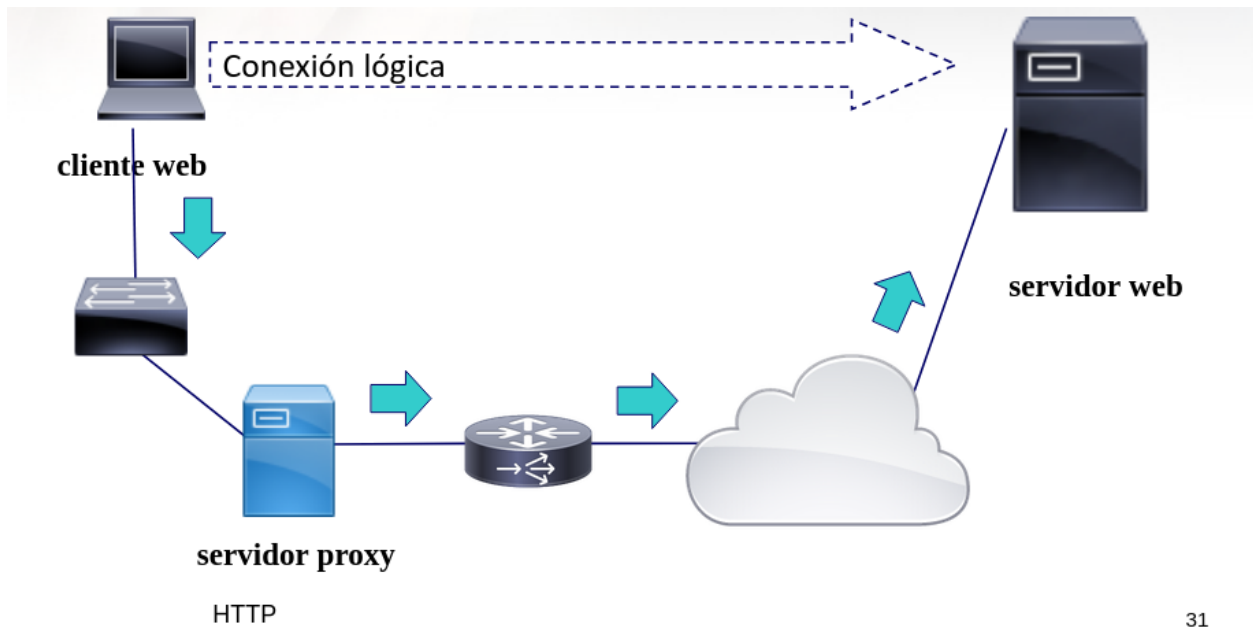


Figure 2: Proxy server

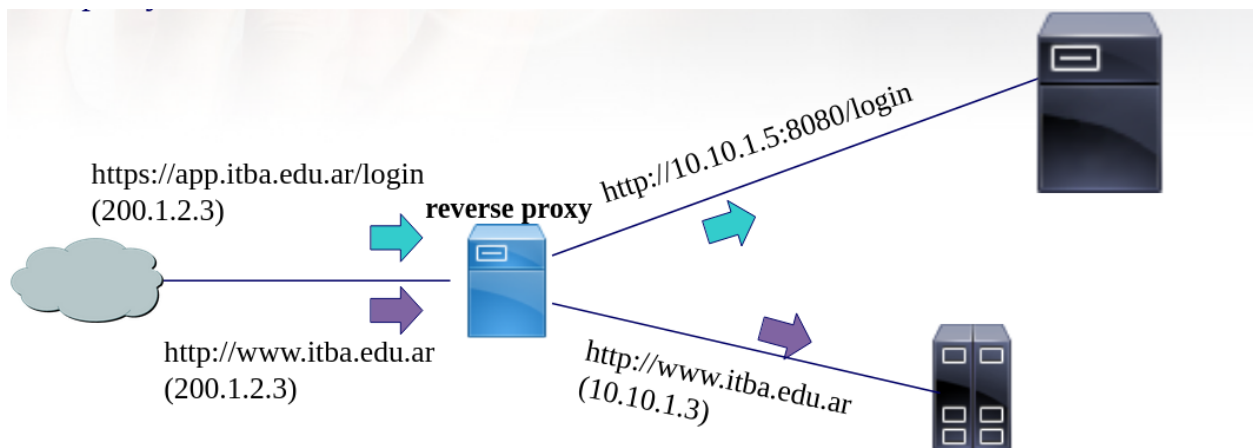


Figure 3: Reverse proxy server

Cookies

La idea es que cuando pida un recurso, en el header esten las cookies. Entonces, cuando el servidor me manda cookies (**set-cookie** res header), a ese servidor en particular, cuando pida un recurso le voy a mandar las cookies(**cookie** req header).

Es un texto de informacion enviado por el servidor y almacenado en el servidor, se usa para mantener un estado entre el cliente y el servidor. Tiene dos tipos de persistencia:

- Session cookie
- Persistent cookie

Tambien hay **third-party** cookie y **secure** cookie.

Un RFC es un pedido a un estandar, se publica, es revisado por un monton de gente. Si se aprueba pasa a ser un estandar.

Curl

A la hora de descargar archivos:

```
curl <url-archivo> --output <archivo>
```

Si lo detengo a la mitad y vuelvo a correr el comando, curl va a empezar de nuevo. Para que continúe con lo que ya tenía hay que agregar **--continue-at**:

```
curl <url-archivo> --output <archivo-nuevo> --continue-at <archivo>
```

Curl asume que el archivo que le das para continuar es la primer parte de la descarga, caso contrario se unen dos archivos y queda algo raro que no sirve.

Es muy buena la documentacion de **Mozilla** para referenciar.

Unidad 3: Resolucion de nombres

Lo que hace HTTP, primero se crea una conexion mediante TCP, luego se pueden mandar los request. Esta conexion se hace a partir de la IP, entonces como se obtienen las IP a partir del domain? Hay dos formas, una usa DNS.

Resolucion de nombres - DNS

Proceso de traducir un nombre de dominio a una direccion IP.

Un **dominio** es una coleccion de computadores que pueden ser accedidas usando un nombre en comun. Un **nombre de dominio** hace referencia al nombre de multiples hosts que son referenciados colectivamente

Las computadoras de un mismo dominio no tienen que estar en el mismo lugar. No tienen nada que ver.

No todos pueden encargarse de la resolucion de nombre, pero si se pueden tener subdominios. Por ejemplo, solo una organizacion da los .com, pero despues yo puedo tener mis propios subdominios como subdominio.ejemplo.com.

No siempre se tiene que acceder a un DNS, podrian estar localmente en `/etc/hosts`, si se encuentra, se usa eso. Ahi se encuentra `localhost`, pues no conviene que tenga una regla especial, es mejor que se comporte como todos los demas. Tambien sirve si queremos bloquear alguna pagina:

127.0.0.1 chatgpt.com

En examen pueden haber preguntas de que codigos se devuelve en cierta situacion, por ejemplo, si mapeo mi pagina web a clarin.com en mi computadora, que pasa cuando el servidor ve que el host del request no es el suyo? Devuelve algun 4XX.

En caso de FQDN, se obtiene su numero de IP del DNS o de `/etc/resolv.conf`. Se pueden especificar:

- `nameserver <direcciones>`
- `domain <nombre>`
- `search <dominios>`
- `sortlist <red\[mascara\]>`
- `option <opcion>`

Los `nameserver` tienen que ya estar especificados por su IP, son los servidores DNS. Cuanto duran las resoluciones en la cache? Depende del `timeout` que te devuelva el servidor DNS junto a la IP.

Linux tiene las utilidades `dig` y `host` para mostrar como la computadora, a partir de un nombre de dominio, encontro la IP(reverse DNS) o al revez.

Si un dia alguna pagina web cambia de IP(cambio de proveedor de internet, cambio de locacion, etc). Entonces, si me quiero conectar, mi computadora va a intentar de acceder a la vieja IP, algo similar va a pasar en el cache de todos los DNS. Entonces, va a haber que notificar a `a.riu.edu.ar` pues es el root server que maneja `edu.ar`, no hace falta notificar al de `ar`.

Como se consiguen los nombres

Es un sistema jerarquico y distribucion para la resolucion de nombres de dominio en direcciones IP(y viceversa). Se lee al revez, empieza la jerarquia mas alta por la derecha. Es **jerarquico y distribuido** porque delega a los de abajo.

El nombre **completo** en realidad es un `.`, es el top-level domain.

Entonces, hay distintos tipos de servidores DNS:

- **Recursivos:** Realizan la consulta completa en nombre del cliente
- **Autorizados:** Contiene la informacion oficial
- **Raiz DNS:** Dominio padre

La distribucion puede ser vista como un arbol invertido donde la raiz hace referencia a servidores de nombre raiz(root name servers).

Los **root name servers** se usan cuando el servidor de nombres local no pueden resolver un nombre. El root name server luego:

1. Contacta a un name server autorizado si no conoce la respuesta
2. Almacena
3. Retorna la respuesta al servidor de nombres local

Hay 13 de estos en total.

Generalmente el **DNS server local** es el router.

Las preguntas y las respuestas de los servidores DNS son UDP.

Cache

Una vez que el servidor de nombres aprende el mapeo, lo almacena en cache.

—

Los servidores raiz DNS son operados por IETF. Cuando alguien obtiene un nombre de dominio es el responsable de mantener, en un servidor DNS primary(antes master), actualizado ese dominio(Completa la idea de lo que ocurre cuando cambio mi IP).

En Unix y Linux DNS es implementado generalmente con **BIND**(Berkeley Internet Name Domain).

Configuracion de servidor DNS

Hay cuatro niveles de configuracion:

- Resolver only systems
- Caching-Only
- Primary(Antes master)
- Secondary(Antes slave)

Registros DNS

DNS es una base de datos distribuida de **resource records**(RR). Los registros son:

- A
- NS
- CNAME
- MX
- AAAA
- SOA
- SRV
- RP
- TXT
- SPF

Los registros asocian el request a distintas cosas, por ejemplo, A devuelve la IP.

SRV es tipo MX, pero mas generico

Formato RR

(name, value, tyype, ttl)

El @ surge para separar el nombre del dominio del nombre del usuario, sino se tornaba confuso.

El tiempo de vida minimo para respuestas negativas se refiere a cuando el servidor tiene que ir a preguntar a otros por un nombre de dominio. Si no lo encuentra, durante las proximas, por ejemplo, 3 horas no volvera intentar a preguntar por el nombre a otros(va a devolver que no lo encontro).

Split-horizon

Un nombre se resuelve con distintas IP dependiendo del origen de la consulta. Por ejemplo, desde el ITBA el servidor de pampero es 10.16.1.103, mientras que desde afuera es 200.5.121.139.

No es si la pregunta se hace desde adentro o desde afuera, sino que donde esta el servidor DNS, si esta afuera me va a dar la IP de afuera por mas que yo este adentro del ITBA.

DNS dinamico

DDNS permite actualizar en forma dinamica un servidor DNS. Introducido en BIND a partir de la version 9. Si cambia la IP, el servidor se va a actualizar automaticamente.

DNS spoofing

Es el arte de lograr que un registro DNS apunte a un IP que no sea el que deberia apuntar. De manera que algun actor malicioso podria crear una copia de la pagina del ITBA y hacer DNS spoofing para que el nombre de dominio del ITBA apunte a su pagina. Esta seria una manera de realizar un ataque de phishing. Por eso es importante preguntar para corroborar la resolucion DNS.

Es comun que muchas empresas sigan usando versiones viejas de software como BIND.

Metodo: Enviar respuesta sin recibir pregunta

Se envia como falsa respuesta el IP de algun nombre de dominio. Se debe verificar que la respuesta corresponda con alguna pregunta. DNS establece que los mensajes de pregunta y respuesta tengan un mismo ID que los identifique.

Metodo: DNS sniffer

Person in the middle(no man in the middle), se obtiene el ID para poder responder con una IP falsa en nombre del servidor DNS.

Metodo: DNS cache poisoning

Inyecta otras resoluciones a una simple pregunta para ensuciar el cache del servidor. Pisa las resoluciones anteriores.

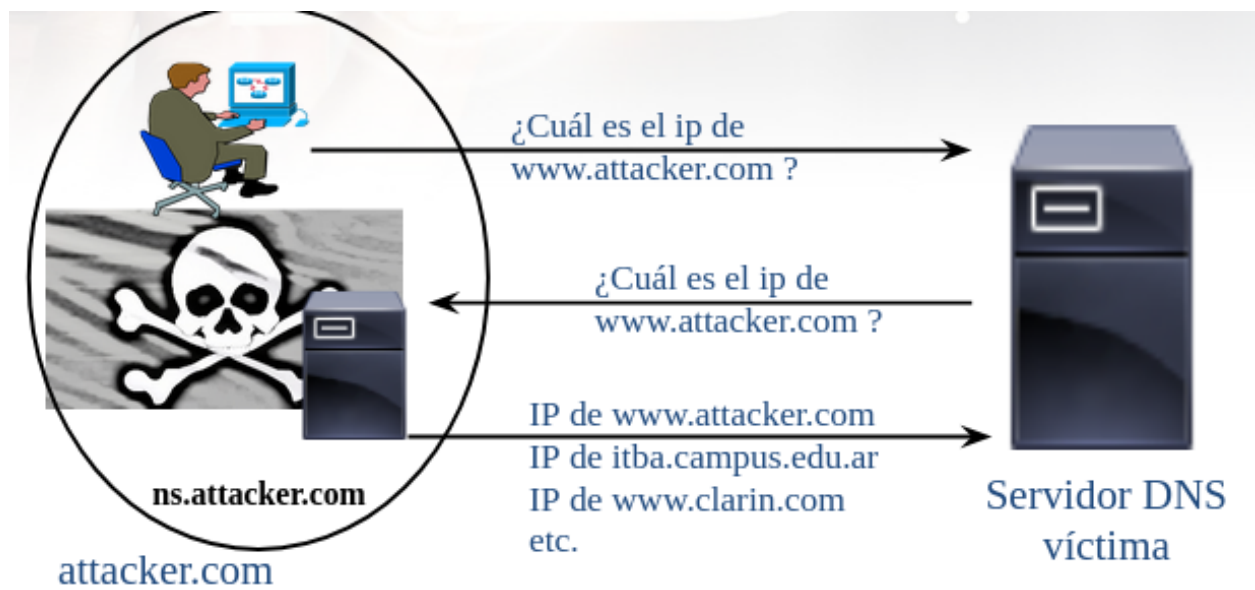


Figure 4: DNS Poisoning